Docker Techniques

Quick revision • Teq 1–11, 15–22

Teq 1	Open Docker Daemon to the World
Teq 2	Running Containers as Daemons
Teq 3	Moving Docker to a Different Partition
Teq 4	socat: Monitor Docker API Traffic
Teq 5	Using Docker in your Browser
Teq 6	Using Ports to Connect to Containers
Teq 7	Allowing Container Communication
Teq 8	Linking Containers for Port Isolation
Teq 9	Setting Up a Local Docker Registry
Teq 10	Finding and Running a Docker Image
Teq 11	Convert a VM to Docker
Teq 15	“Save Game”: Cheap Source Control
Teq 16	Docker Tagging
Teq 17	Share Images on Docker Hub
Teq 18	Referring to a Specific Image in Builds
Teq 19	“Save Game”: Winning at 2048
Teq 20	Injecting Files using ADD
Teq 21	Rebuilding without the Cache
Teq 22	Busting the Cache

Teq 1 • Open Docker Daemon to the World

Make the daemon listen on TCP port 2375 so other machines can use the Docker API.

1. Ubuntu: check Docker

```
sudo systemctl status docker
```

2. Open the service file

```
sudo nano /lib/systemd/system/docker.service
```

3. Comment out this line

```
ExecStart=/usr/bin/dockerd -H fd://
--containerd=/run/containerd/containerd.sock
```

4. Add this line

```
ExecStart=/usr/bin/dockerd -H=fd://
-H=tcp://0.0.0.0:2375
```

5. Reload + restart

```
sudo systemctl daemon-reload ; systemctl restart
docker
```

6. Find IP, then open in browser

```
sudo ifconfig
http://<IP>:2375/images/json
```

7. Non-Ubuntu: /etc/docker/daemon.json

```
{
  "hosts": [
    "tcp://0.0.0.0:2375",
    "unix:///var/run/docker.sock"
  ]
}
```

8. /etc/systemd/system/override.conf

```
[Service]
ExecStart=
ExecStart=/usr/bin/dockerd
```

9. Restart

```
sudo systemctl restart docker
```

Remember

- Default: daemon listens only on /var/run/docker.sock
- -H = where the daemon listens; 0.0.0.0 = all interfaces
- Port 2375 = no auth, no encryption, so it is VERY insecure

Teq 2 • Running Containers as Daemons

Run a container in the background like a service using -d.

1. Create folder + Dockerfile

```
mkdir secondfile
cd secondfile
sudo nano Dockerfile
```

2. Dockerfile

```
FROM alpine
RUN apk add --update redis
CMD ["redis-server"]
```

3. Build + check

```
sudo docker build -t secondfile .
sudo docker images
```

4. Foreground run (blocks terminal, Ctrl+C to stop)

```
docker run secondfile
```

5. Detached run (prompt returns)

```
sudo docker run -d secondfile
```

6. Check

```
sudo docker ps
```

Remember

- RUN = build time, CMD = container start
- -d = detached / background
- docker logs <name> shows output; docker rm cleans up

Teq 3 • Moving Docker to a Different Partition

Change where Docker stores images/containers (default /var/lib/docker).

1. Stop Docker

```
sudo systemctl stop docker.service
```

2. Start daemon with new data root

```
sudo dockerd --data-root /home/deepika/mydocker
```

3. Start Docker service

```
sudo systemctl start docker.service
```

4. Check the folder

```
cd /home/deepika/mydocker
```

Remember

- Book/slides use: dockerd -g /home/dockeruser/mydocker
- -g is old; modern flag is --data-root
- Permanent: "data-root" in /etc/docker/daemon.json

Note: Starts EMPTY. Old images are not copied unless you copy /var/lib/docker yourself.

Teq 4 • socat: Monitor Docker API Traffic

Put a proxy socket between client and daemon to see the raw HTTP calls.

1. Start proxy

```
sudo socat -v
UNIX-LISTEN:/tmp/dockerapi.sock,fork
UNIX-CONNECT:/var/run/docker.sock
```

2. New terminal: send a request

```
curl --unix-socket /var/run/docker.sock
http://localhost/images/json
```

3. Remove socket if it exists

```
sudo rm -f /tmp/dockerapi.sock
```

4. Kill

```
sudo pkill socket
```

Remember

- Every docker command = an HTTP request to the daemon
- -v prints traffic; fork handles many connections
- Needs root/sudo

Note: To actually SEE traffic, curl /tmp/dockerapi.sock (the proxy). To kill, use: pkill socat.

Teq 5 • Using Docker in your Browser

Use a browser as a Docker client (same setup as Teq 1).

1. Open daemon on port 2375 (Teq 1), then visit

```
http://localhost:2375/images/json
http://localhost:2375/containers/json
http://localhost:2375/info
```

Remember

- Slide solution: open port + CORS enabled
- Serve the docker-terminal page in a web server
- Browser -> terminal page -> daemon -> container

Teq 6 • Using Ports to Connect to Containers

Make a container service reachable from the host with -p.

1. Pull image

```
docker pull wordpress
```

2. Map host 10001 -> container 80

```
docker run -d -p 10001:80 --name container1
wordpress
```

3. Verify in browser

```
http://localhost:10001
```

Remember

- -p HOST:CONTAINER (always host first)
- EXPOSE only declares; -p actually publishes

Teq 7 • Allowing Container Communication

Let containers talk to each other using a user-defined network.

1. List networks (bridge, host, none)

```
docker network ls
```

2. Create network

```
docker network create my_network
```

3. Two containers (separate shells)

```
docker run --name container1 -it firstfile:latest
docker run --name container2 -it firstfile:latest
```

4. Connect both

```
docker network connect my_network container1
docker network connect my_network container2
```

5. Inspect

```
docker network inspect my_network
```

6. Inside both containers

```
apt-get update
apt-get install iputils-ping
```

7. Ping by name

```
docker exec -it container1 ping container2
```

Remember

- User-defined network = built-in DNS (reach by container NAME)
- Default bridge has no name resolution

Teq 8 • Linking Containers for Port Isolation

Connect containers WITHOUT user-defined networks using --link.

1. Pull images

```
docker pull mysql/mysql-server:5.6
docker pull nimmis/apache-php5
```

2. Run MySQL first

```
docker run -p 3306:3306 --name mysql -e
MYSQL_ROOT_PASSWORD=root123 -d
mysql/mysql-server:5.6
```

3. Run Apache linked to MySQL

```
docker run -tid -p 80:80 --name apache2 --link
mysql nimmis/apache-php5
```

4. Check + inspect

```
docker ps
docker inspect mysql
docker inspect apache2
```

5. Enter Apache, ping mysql

```
docker exec -ti apache2 bash
ping mysql
```

Remember

- --link adds 'mysql' to /etc/hosts of apache2
- -e sets environment variables
- --link is legacy; networks are the modern way

Teq 9 • Setting Up a Local Docker Registry

Host your own images locally.

1. Run registry

```
docker run -d -p 5000:5000 -v
$HOME/registry:/var/lib/registry registry:2
```

2. Use it

```
docker tag myimg localhost:5000/myimg
docker push localhost:5000/myimg
docker pull localhost:5000/myimg
```

Remember

- registry:2 = official registry image, port 5000
- -v keeps images on disk if the container dies
- Image name decides where it is pushed

Teq 10 • Finding and Running a Docker Image

Find a tool on Docker Hub and try it.

1. Search

```
docker search node
```

2. Pull

```
docker pull node
```

3. Run

```
docker run -it node
```

Remember

- search = find, pull = download, run = start
- Prefer OFFICIAL [OK] images
- docker run auto-pulls if image is missing

Teq 11 • Convert a VM to Docker

Turn a whole machine's filesystem into an image (export/import).

1. Run nginx

```
docker run -d -p 80:80 --name nginx nginx
docker ps
```

2. Export filesystem to tar

```
docker export nginx > mynginx.tar
```

3. Import tar as image

```
docker import - mynginx < mynginx.tar
docker images
ls -l
```

4. Run it (must give command)

```
docker run --name container_nginx -d
mynginx:latest nginx -g 'daemon off;'
```

5. Check

```
docker ps
```

Remember

- export/import = container FILESYSTEM (flattened, 1 layer)
- Loses CMD/ENV/ports, so give command yourself
- save/load = images WITH layers & metadata

Teq 15 • “Save Game”: Cheap Source Control

Snapshot a container's state as a new image with docker commit.

1. Check

```
docker images
docker container ls
```

2. Start Ubuntu interactively

```
docker run -it ubuntu:latest bin/bash
```

3. Inside: install a tool

```
apt-get update
apt-get install nmap
nmap --version
```

4. NEW terminal: commit

```
docker commit [CONTAINER_ID] [new_image_name]
```

5. Exit + verify

```
exit
docker ps
docker images
```

Remember

- commit = container -> image (the 'save')
- Container ID = hex in the prompt (root@3f2a1b..)
- Quick but not reproducible; Dockerfile is better

Teq 16 • Docker Tagging

Give an image a name/version.

1. List

```
sudo docker images
```

2. Tag

```
sudo docker tag [IMAGE_ID] [new_image_name]
```

Remember

- Format: [registry/][user/]repo[:tag]
- Tag = label, NOT a copy (same IMAGE ID)
- Default tag = latest

Teq 17 • Share Images on Docker Hub

Push your image to Docker Hub so anyone can pull it.

1. Login

```
docker login
```

2. Get an image

```
docker images
docker pull ubuntu:latest
```

3. Make changes (inside a container), then commit

```
apt-get update
apt-get install nmap
```

4. Tag with your username

```
docker tag local-image:tagname
username/repository:tagname
```

5. Push

```
docker push username/repository:tagname
```

Remember

- Flow: login -> tag -> push
- Hub needs username/repo name, or push is denied

Teq 18 • Referring to a Specific Image in Builds

Pin the base image in FROM so builds are reproducible.

1. Examples

```
FROM ubuntu # latest, changes
FROM ubuntu:22.04 # pinned version
FROM ubuntu@sha256:<digest> # exact
```

Remember

- Same process as 'Writing a Dockerfile'
- latest is a moving target; pin tag or digest

Teq 19 • “Save Game”: Winning at 2048

Back up a container as a tar and restore it later.

1. Export (save)

```
docker export Container_ID > filename.tar
```

2. Import as image

```
docker import - [Container_Name] < filename.tar
```

3. Verify

```
docker images
```

4. Pull an old backup

```
docker pull filename-backup:latest
```

5. Run restored (load)

```
docker run -it my-backup:latest
```

Remember

- export = save file, import = restorable image
- Imported image has no CMD; add /bin/sh

Note: Pull only works if the backup was pushed to a registry first.

Teq 20 • Injecting Files using ADD

Copy host files into the image at build time.

1. Create sample.txt in the Dockerfile folder

2. Dockerfile

```
FROM ubuntu:latest
RUN apt-get -y update
ADD sample.txt .
```

3. Build + list

```
docker build -t sample-image .
docker images
```

4. Run + verify

```
docker run -it sample-image bash
ls
```

Remember

- ADD auto-EXTRACTS local .tar files; COPY never does
- File must be inside the build context
- Plain files: prefer COPY

Teq 21 • Rebuilding without the Cache

Force every step to run fresh.

1. Build with no cache

```
docker build --no-cache .
```

Remember

- Cache reuses unchanged layers (fast but can be stale)
- --no-cache ignores ALL cache

Teq 22 • Busting the Cache

Comparison of builds with and without cache; rebuild from a chosen step.

1. Compare

```
docker build -t img . # all CACHED, fast
docker build --no-cache -t img . # all run, slow
```

2. Bust from a step with ARG

```
ARG CACHEBUST=1

docker build --build-arg CACHEBUST=$(date +%s) -t
img .
```

Remember

- First changed step + EVERYTHING after it rebuilds
- Change a COPY/ADD file or instruction to bust
- Put rarely-changing steps at the top